

# Bản trình chiếu: Tối ưu tìm kiếm sâu qua bài chia hình vuông

Jin Kai

2003

*Nguồn gốc: Exploring optimization techniques in depth-first search: from the square dissection problem*

IOI China candidate team papers / 2003 / Jin Kai / Jin Kai.ppt

## 1 Bài toán chia hình vuông

Bài toán yêu cầu chia hình vuông lớn gồm  $n \times n$  ô đơn vị thành một số hình vuông nhỏ hơn, mỗi hình có cạnh nguyên, sao cho số hình vuông nhỏ là ít nhất. Ràng buộc trong slide là

$$1 \leq n \leq 32,$$

môi trường lập trình là *FreePascal*, bộ nhớ khả dụng 64MB. Ví dụ  $n = 7$  có một phương án tối ưu dùng 9 hình vuông nhỏ.

Nếu  $n$  chẵn, đáp án hiển nhiên không quá 4: chia hình vuông thành bốn hình vuông cạnh  $n/2$ . Slide xem đây là trường hợp đơn giản. Khi  $n$  lẻ, khó nhận ra quy luật toán học trực tiếp, nên trọng tâm chuyển sang tìm kiếm sâu có cắt tỉa.

## 2 Một cận trên bằng quy hoạch động hình chữ nhật

Trước khi tìm kiếm, slide xét quy hoạch động trên hình chữ nhật. Gọi

$$f_{i,j}$$

là số hình vuông ít nhất để chia hình chữ nhật  $i \times j$ . Chuyển trạng thái tự nhiên là cắt hình chữ nhật bởi một đường thẳng ngang hoặc dọc:

$$f_{i,j} = \min \left\{ \min_k (f_{i,k} + f_{i,j-k}), \min_k (f_{k,j} + f_{i-k,j}) \right\}.$$

Cách này chỉ cho một cận trên. Với  $n = 7$ , nó cho kết quả 10, không phải tối ưu, vì phương án tối ưu không nhất thiết bị tách được bởi một đường thẳng duy nhất thành hai phần độc lập. Tuy vậy giá trị  $f_{n,n}$  thường gần đáp án tối ưu, nên có thể dùng làm nghiệm ban đầu để giới hạn độ sâu tìm kiếm.

## 3 Tìm kiếm ban đầu và cắt tỉa

Đối tượng tìm kiếm là vị trí và cạnh của từng hình vuông nhỏ. Một thứ tự đơn giản là đánh số ô  $(i, j)$  bằng  $(i-1)n + j$ , mỗi bước lấy ô chưa phủ có số nhỏ nhất làm góc trái trên của hình vuông tiếp theo, rồi thử các cạnh khả dĩ. Cách này đúng nhưng quá chậm.

Các cắt tỉa đầu tiên trong slide gồm:

- cắt tia đối xứng: quy định cạnh hình vuông ở góc trái trên không lớn hơn các hình ở ba góc còn lại, và hình ở góc phải trên không nhỏ hơn hình ở góc trái dưới;
- cắt tia tổng bình phương: nếu diện tích còn lại là  $i$ , dùng  $Sum_i$  là số ít nhất các số nguyên cần có để tổng bình phương bằng  $i$ , từ đó lấy làm cận dưới số hình vuông còn phải thêm.

Tuy nhiên, với thứ tự quét ô đơn giản, vùng đã phủ thường lồi lõm khó kiểm soát, làm cận dưới yếu và số nhánh lớn.

## 4 Thứ tự tìm kiếm theo đường chia

Slide đưa ra khái niệm “hình chữ nhật dạng gần chữ nhật”. Trực giác là biên giữa phần đã phủ và phần chưa phủ có thể xem như một đường chia đi từ góc trái dưới đến góc phải trên, chỉ đi sang phải hoặc đi lên. Hình vuông lớn ban đầu là một trạng thái như vậy; với mỗi trạng thái, có thể chọn một phần lõm và tách ra một hình vuông mà vẫn giữ dạng này.

Trong tìm kiếm mới, mỗi bước chọn góc trái trên của một phần lõm làm góc trái trên của hình vuông, và cạnh hình vuông không vượt quá bề rộng phần lõm. Nhờ đó vùng đã phủ luôn có cấu trúc được kiểm soát. Để tránh đếm lại cùng một phương án, slide đặt thêm quy tắc thứ tự: hình vuông trước không được nằm hoàn toàn phía trên hình vuông sau.

Thứ tự mới tạo ra cận dưới mạnh hơn. Nếu trạng thái hiện tại có  $x$  phần lồi, nó cần ít nhất  $x$  hình vuông nữa, vì ô ở góc phải dưới của mỗi phần lồi phải thuộc về một hình vuông khác nhau. Thực nghiệm trong slide cho biết cận này thường chính xác hơn cận  $Sum_i$ . Kết hợp thêm đối xứng qua đường chéo trái trên–phải dưới, thời gian trường hợp chậm  $n = 31$  giảm còn khoảng 20 giây trên máy thử nghiệm được ghi trong slide.

## 5 Tiên xử lý giá trị trạng thái

Slide tiếp tục tối ưu bằng ý tưởng biến một số nút trong cây tìm kiếm thành lá. Nếu đã dùng  $x$  hình vuông và biết vùng chưa phủ cần ít nhất  $y$  hình vuông nữa, thì mọi lời giải đi qua nút đó có giá trị  $x + y$ ; không cần mở rộng nút.

Vấn đề là biểu diễn trạng thái vùng chưa phủ. Một cách trực tiếp dùng bộ

$$(a_1, a_2, \dots, a_n), \quad a_i \geq a_{i+1},$$

trong đó  $a_i$  là số ô chưa phủ trên một hàng tương ứng. Cách này tốn  $n$  byte cho mỗi trạng thái và so sánh hai trạng thái mất  $O(n)$ .

Để mã hóa gọn hơn, đặt

$$b_i = a_i - a_{i+1} \quad (0 \leq i \leq n), \quad a_0 = n, \quad a_{n+1} = 0.$$

Khi đó  $b_i \geq 0$  và

$$\sum_i b_i = n.$$

Số bộ  $b$  là số cách đặt  $n$  vật không phân biệt vào  $n + 1$  hộp có nhãn, tức  $\binom{2n}{n}$ , với  $n = 31$  vẫn không vượt quá  $10^{18}$  theo ghi chú của slide. Vì  $a$  và  $b$  tương ứng một-một, có thể dùng một số nguyên kiểu comp để mã hóa trạng thái.

Slide phác thảo bảng

$$Tot_{i,j}$$

là số dãy thỏa  $j \geq a_i \geq a_{i+1} \geq \dots \geq a_n \geq 0$ , dùng để chuyển qua lại giữa bộ  $a$  và số nguyên. Các đoạn mã trong slide chỉ là mẫu chuyển đổi chỉ số: cộng dồn  $Tot[i, a[i] - 1]$  khi mã hóa, và giảm  $a[i]$  theo  $Tot$  khi giải mã; bản dịch này không chép lại đầy đủ mã khuôn.

## 6 BFS trạng thái nhỏ và bảng băm

Sau khi trạng thái được mã hóa bằng một số nguyên, ta có thể tính trước giá trị tối ưu của một số trạng thái. Nếu xem mỗi trạng thái là một đỉnh, và thêm một hình vuông tạo ra một cạnh có độ dài 1, thì giá trị  $f_i$  của trạng thái  $i$  là khoảng cách ngắn nhất từ trạng thái rỗng tới  $i$ . Vì mọi cạnh cùng trọng số, có thể dùng BFS từ trạng thái ban đầu và mở rộng đến khi có đủ nhiều trạng thái.

Mỗi trạng thái sinh ra được đưa vào bảng băm để tra cứu khi DFS chính đang chạy. Nếu trạng thái hiện tại có trong bảng, giá trị đã biết cho phép cập nhật nghiệm tốt nhất và quay lui ngay. Để tránh tra cứu vô ích ở hầu hết các nút, slide chỉ tính các trạng thái có  $f_i \leq c$ , với  $c$  là hằng số tự chọn. Khi đã dùng  $x$  hình vuông và  $x + c + 1$  không còn tốt hơn nghiệm hiện tại, mới cần tra bảng:

- nếu không tìm thấy, vùng còn lại cần ít nhất  $c + 1$  hình vuông, nên nhánh này không thể cải thiện nghiệm;
- nếu tìm thấy  $f_i$ , cập nhật nghiệm bằng  $x + f_i$  rồi quay lui.

Nhờ tiền xử lý này, độ sâu tìm kiếm giảm được khoảng  $c$  tầng. Kết hợp với các cắt tỉa trước đó, trường hợp  $n = 31$  trong bảng thời gian của slide chỉ còn hơn 4 giây.

## 7 Tổng kết

Quá trình giải trong slide gồm bốn bước: dùng DP hình chữ nhật để lấy cận trên, thêm các cắt tỉa đối xứng và diện tích, đổi thứ tự tìm kiếm theo cấu trúc đường chia, rồi mã hóa trạng thái để tiền xử lý một phần cây tìm kiếm.

Thông điệp chung là thứ tự và đối tượng tìm kiếm là nền tảng của hiệu quả, nhưng cắt tỉa đơn thuần không làm thay đổi bản chất độ phức tạp. Khi bộ nhớ cho phép, có thể hy sinh không gian để tính trước giá trị của các trạng thái nhỏ, từ đó giảm độ sâu DFS. Bước khó nhất của tối ưu này là biểu diễn trạng thái đủ gọn để tra cứu nhanh.